קטע קוד A

```
class A
{
  static A* first;
  static A* last;
  A* prev, * next;
public:
  A(): prev(last), next(nullptr) {
    this->printSelf();
    if (last) last->next = this;
    last = this;
    if (!first) first = this;
  }

  ~A() {
    if (prev) prev->next = next;
    if (next) next->prev = prev;
    if (first == this) first = next;
    if (last == this) last = prev;
  }

  static void printAll() {
    if (first) first->printAllFromThis();
  }

  virtual void printSelf() const {
    cout << "A" << endl;
  }

  friend ostream& operator << (ostream& out, const A& a)
  {
    a.printSelf();
    return out;
  }

  void printAllFromThis() {
    printSelf();
    if (next) {
      next->printAllFromThis();
    }
    else {
      cout << endl;
    }
  }

```

```
45.   template<class Function>
46.   static A* search(Function condition) {
47.     for (A* elem = first; elem != nullptr; elem = elem->next) {
48.       if (condition(*elem)) return elem;
49.     }
50.     return nullptr;
51.   }
52.
53.   virtual bool operator > (int x) const = 0;
54. };
55.
56. class B :public A {
57.   int* i = nullptr;
58. public:
59.   B(int _i) : i(new int(_i)) {
60.     cout << *i << endl;
61.   }
62.
63.   ~B() {
64.     delete i;
65.   }
66.
67.   virtual void printSelf() const override {
68.     cout << *i << ' ';
69.   }
70.
71.   bool operator > (int x) const {
72.     return *i > x;
73.   }
74. };
75.
76. A* A::first = nullptr;
77. A* A::last = nullptr;
78.
79. int main()
80. {
81.   B b1 = 5;
82.   const B b2 = 15;
83.   B b3 = 25;
84.
85.   // B b4 = b3;
86.   // A::printAll();
87.
88.   // cout << b1 << endl;
89.
90.   //cout << *(A::search([](const A& element) {return element > 20; }));
91.   return 0;
92. }
```

קטע קוד B – לבחינה פרונטלית בלבד

```
template<class E> class MyList {
    class Node {
        friend class MyList<E>;
        E data;
        Node* next = nullptr;
    }; // end of class Node
    Node* head = new Node;

public:
    void add(const E& data) {
        Node* node = new Node;
        node->data = data;
        node->next = head->next;
        head->next = node;
    }
    ~MyList() {
        delete head;
    }
    template<class Function>
    void apply(Function f) const {
        Node* node = head->next;
        while (node != nullptr) {
            f(node->data);
            node = node->next;
        }
    }
    MyList<E>& operator+= (const E& value_to_add) {
        Node* node = head->next;
        while (node != nullptr) {
            node->data = node->data + value_to_add;
            node = node->next;
        }
        return *this;
    }
};
```

נתון: הקוד תקין וניתן לכתוב עבורו main תקין שיעשה שימוש בכל הפונקציות שהוגדרו.
בעמוד הבא מופיעים קטעי main רלבנטיים.

להלן מספר קטעי main וקוד עזר (כל main עומד בפני עצמו).

```
 1. // M A I N - 1
 2. int main() {
 3.     MyList<int> m1;
 4.     m1.add(10);
 5.     m1.add(20);
 6.     m1 += 30;
 7.     m1.apply([](const auto& val) {cout << val << endl; });
 8.     return 0;
 9. }
10.
11.
12. // M A I N - 2
13. struct A {
14.     virtual void print() const {
15.         cout << "A" << endl;
16.     }
17. };
18.
19. struct B : public A {
20.     void print() const {
21.         cout << "B" << endl;
22.     }
23. };
24.
25. int main() {
26.     A a;
27.     B b;
28.
29.     MyList<A*> m2;
30.     m2.add(&a);
31.     m2.add(&b);
32.     m2.apply([](auto val) { val->print(); });
33.
34.     MyList<A> m3;
35.     m3.add(a);
36.     m3.add(b);
37.     // m3 += a;
38.     m3.apply ([](auto val) { val.print(); });
39.
40.     return 0;
41. }
```

קוד C – לשאלות רשות

```
class A {
    static A* first;
    static A* last;
    A *prev, *next;
public:
    A(): prev(last), next(nullptr) {
        if(last) last->next = this;
        last = this;
        if(!first) first = this;
    }
    A(const A&): A() {}
    A(A&& a): prev(a.prev), next(a.next) {
        if(&a == first) first = this;
        if(&a == last) last = this;
        if(a.prev) a.prev->next = this;
        if(a.next) a.next->prev = this;
    }
    A& operator=(const A&) {return *this;}
    ~A() {
       // print();
        if(prev) prev->next = next;
        if(next) next->prev = prev;
        if(first == this) first = next;
        if(last == this) last = prev;
    }
    static void printAll() {
        if(first) first->print();
        else cout << "list is empty";
        cout << endl;
    }
    void print()const {
        printSelf();
        if(next) next->print();
    }
    virtual void printSelf()const = 0;
};

class B: public A {
    int i;
public:
    B(int _i): i(_i) {}
    // ~B() { printAll(); }
    virtual void printSelf()const override {
        cout << i << ' ';
    }
};
```

```
class C: public B {
public:
    C(): B(7) {}
    ~C() { printAll(); }
    virtual void printSelf()const override {
        cout << "C: ";
        B::printSelf();
    }
};

A* A::first = nullptr;
A* A::last = nullptr;

int main() {
  B b1 = 5;
  B b2 = b1;
  b2 = 12;
  B b3 = std::move(b1);
  b3 = 99;
  C c1;
  cout << "end of main" << endl;
}
```

בהצלחה !